Zip Code Group Project 2.0 — Testing Document

Team 5

St Cloud State University

CSCI 331 - Software Systems — Spring 2025

Dr. Anda, Andrew

March 22nd, 2025

*Written with the utmost dedication and **last-minute** inspiration.*

…

Table of Contents

---

1. Introduction

- **Objective:** This project handles CSV and LRF records, reads and writes header records, and allows searching and reporting functionality using the given buffers and classes.

- **Modules Tested:**
    - CSVBuffer
    - LRFBuffer
    - HeaderRecordBuffer
    - StateExtremesReport
    - CSVToLengthIndicated

- **Purpose of Testing:** Ensure each function operates correctly, handles input/output as expected, and interacts properly with the different file types.

---

2. Test Cases and Results

2.1. Test Case: Printing CSV Records

- **Function:** `printCSVRecords(CSVBuffer& buffer)`
- **Purpose:** Validate that CSV records are correctly read and printed from the given CSV file.
- **Steps:**
  - Select “Print CSV Records” by inputting 1
  - Input a valid CSV file.
  - Ensure the correct headers are printed.
  - Validate that each record is printed with corresponding column headers.
- **Expected Output:**
  - Column headers are displayed.
  - Each CSV record is printed alongside its respective headers.
- **Results:** **The test passed successfully.**

```
1. Print CSV Records
2. Print LRF Records
3. Convert CSV to LRF
4. Read Header Record
5. Write (Update) Header Record
6. Run State Extremes Report
7. Search for Zip Code Records by Index
Enter your choice: 1
Enter the CSV file name to print records from: us_postal_codes.csv
...
Zip_Code = 99928 | Place_Name = Ward Cove | State = AK | County = Ketchikan
Gateway | Lat = 55.3954 | Long = -131.6754 |
Zip_Code = 99929 | Place_Name = Wrangell | State = AK | County =
Wrangell-Petersburg (C | Lat = 56.4335 | Long = -132.3529 |
Zip_Code = 99950 | Place_Name = Ketchikan | State = AK | County = Ketchikan
Gateway | Lat = 55.3422 | Long = -131.6478 |
```

2.2. Test Case: Convert CSV to LRF

- **Function:** `convertCSVToLRF(const std::string& csvFilename, const std::string& lrfFilename)`

- **Purpose:** To ensure that the CSV file is correctly converted into an LRF file, maintaining accurate records and structure.

- **Steps:**

    - Select “Convert CSV to LRF” by inputting 3
    - Input a valid CSV file:
        - Provide the us_postal_codes.csv file as the input.
    - After the conversion, check if the LRF file is generated and contains the expected structure based on the CSV file.
    - Ensure that each CSV record is correctly mapped to the corresponding LRF record, with fields such as `Zip_Code`, `Place_Name`, `State`, `County`, `Lat`, and `Long` correctly populated in the LRF file.

- **Expected Output:**

    - The LRF file `us_postal_codes.lrf` is successfully generated.

    - Each CSV record is correctly converted into an LRF record with appropriate field names and values.

    - No data loss or corruption occurs during the conversion process.

- **Results:** **The test passed successfully.**

```
1. Print CSV Records
2. Print LRF Records
3. Convert CSV to LRF
4. Read Header Record
5. Write (Update) Header Record
6. Run State Extremes Report
7. Search for Zip Code Records by Index
Enter your choice: 3
Enter the CSV file name to convert to LRF: us_postal_codes
Enter the LRF file name (without extension) to save as .lrf (Leave empty to
use the same name as the CSV):
Conversion complete. Structured file created as 'us_postal_codes.lrf'.
```

2.3. Test Case: Printing LRF Records

- **Function:** `printLRFRecords(LRFBuffer& buffer)`
- **Purpose:** Validate that LRF records are correctly read and printed from the given LRF file.
- **Steps:**
  - Select “Print LRF Records” by inputting 2
  - Input a valid LRF file.
  - Ensure each field in the record is printed as expected.
- **Expected Output:**
  - LRF records are printed field by field, with each field displayed correctly.
- **Results:** **The test passed successfully.**

```
1. Print CSV Records
2. Print LRF Records
3. Convert CSV to LRF
4. Read Header Record
5. Write (Update) Header Record
6. Run State Extremes Report
7. Search for Zip Code Records by Index
Enter your choice: 2
Enter the LRF file name to print records from: us_postal_codes.lrf
...
99928 | Ward Cove | AK | Ketchikan Gateway | 55.3954 | -131.6754 |
99929 | Wrangell | AK | Wrangell-Petersburg (C | 56.4335 | -132.3529 |
99950 | Ketchikan | AK | Ketchikan Gateway | 55.3422 | -131.6478 |
```

2.4. Test Case: Read Header Record

- **Function:** `readHeaderRecord(const std::string& filename)`
- **Purpose:** Ensure that header information from an LRF file is correctly read and displayed.
- **Steps:**
  - Select “Read Header Record” by inputting 4
  - Input a valid LRF file.
  - Check that the file type, version, and record length field size are correctly displayed.
  - Verify that field definitions are printed correctly.
- **Expected Output:**
  - Correct header information (file type, version, field count, etc.) is displayed.
- **Results:** **The test passed successfully.**

```
1. Print CSV Records
2. Print LRF Records
3. Convert CSV to LRF
4. Read Header Record
5. Write (Update) Header Record
6. Run State Extremes Report
7. Search for Zip Code Records by Index
Enter your choice: 4
Enter the LRF file name to read the header from: us_postal_codes

--- Header Record ---
File Type: ûZIPCODE_DATA
Version: 1.0
Record Length Field Size: 4
Size Format Type: binary
Primary Key Index File Name: us_postal_codes.idx
Record Count: 40933
Field Count: 6
Field Definitions:
  Zip_Code,string,1
  Place_Name,string,0
  State,string,0
  County,string,0
  Lat,string,0
  Long,string,0
----------------------
```

2.5. Test Case: Write Header Record

- **Function:** `writeHeaderRecord(const std::string& filename)`
- **Purpose:** Ensure that the header record is updated correctly and the changes are written to the LRF file.
- **Steps:**
  - Select “Write (Update) Header Record” by inputting 5
  - Input a valid LRF file.
  - Display the current header details.
  - Input new values for file type and version.
  - Verify that the updated header is written to the file.
- **Expected Output:**
  - Updated file type and version values are correctly written to the file.
- **Results:** **The test passed successfully.**

```
1. Print CSV Records
2. Print LRF Records
3. Convert CSV to LRF
4. Read Header Record
5. Write (Update) Header Record
6. Run State Extremes Report
7. Search for Zip Code Records by Index
Enter your choice: 5
Enter the LRF file name to update the header: us_postal_codes

Current File Type: ûZIPCODE_DATA
Current Version: 1.0
Enter new File Type: ûZIPCODE_Test
Enter new Version: 1.1
Header record updated successfully.
```

2.6. Test Case: Run State Extremes Report

- **Function:** `runStateExtremesReport(const std::string& filename)`
- **Purpose:** Test that the state extremes report is generated correctly for a given LRF file.
- **Steps:**
  - Select “Run State Extremes Report” by inputting 6
  - Input a valid LRF file.
  - Verify that the report outputs the expected state extremes (values, formatting).
- **Expected Output:**
  - A report showing the state extremes is printed to the console.
- **Results**: **The test passed successfully.**

```
1. Print CSV Records
2. Print LRF Records
3. Convert CSV to LRF
4. Read Header Record
5. Write (Update) Header Record
6. Run State Extremes Report
7. Search for Zip Code Records by Index
Enter your choice: 6
Enter the LRF file name to run state extremes report from: us_postal_codes

State   | Easternmost Zip | Westernmost Zip | Northernmost Zip |
Southernmost Zip
-------------------------------------------------------------------------
AA      34034           34034           34034           34034
...
WA      99402           98350           98231           98671
WI      54246           54082           54850           53102
WV      25425           25555           26050           24884
WY      82082           83120           82423           82321
```

2.7.1. Test Case: Search us_postal_codes.lfr Zip Code Records by Index

- **Function:** `searchZipCodeRecords(const std::string& lrfFilename)`
- **Purpose:** Ensure that Zip Code records can be searched by using an index file.
- **Steps:**
  - Select “Search for Zip Code Records by Index” by inputting 7
  - Input a valid LRF file.
  - Provide one or more Zip Code flags as input.
  - Verify that the correct record(s) is fetched and displayed.
- **Expected Output:**
  - The program correctly fetches and prints the records corresponding to the Zip Codes.
- **Results**: **The test passed successfully.**

```
1. Print CSV Records
2. Print LRF Records
3. Convert CSV to LRF
4. Read Header Record
5. Write (Update) Header Record
6. Run State Extremes Report
7. Search for Zip Code Records by Index
Enter your choice: 7
Enter the LRF file name to search in: us_postal_codes
Enter Zip Code flags (e.g., -Z56301 -Z12345), separated by spaces: -Z56301
-Z56303 -Z12345
Record for Zip Code 56301:
Zip_Code: 56301 | Place_Name: Saint Cloud | State: MN | County: Stearns |
Lat: 45.541 | Long: -94.1819 |
Record for Zip Code 56303:
Zip_Code: 56303 | Place_Name: Saint Cloud | State: MN | County: Stearns |
Lat: 45.5713 | Long: -94.2036 |
Record for Zip Code 12345:
Zip_Code: 12345 | Place_Name: Schenectady | State: NY | County: Schenectady
| Lat: 42.8142 | Long: -73.9396 |
```

2.7.2. Test Case: Search us_postal_codes_ROWS_RANDOMIZED.lfr Zip Code Records by Index

- **Function:** `searchZipCodeRecords(const std::string& lrfFilename)`
- **Purpose:** Ensure that Zip Code records can be searched by using an index file.
- **Steps:**
  - Select “Search for Zip Code Records by Index” by inputting 7
  - Input a valid LRF file.
  - Provide one or more Zip Code flags as input.
  - Verify that the correct record(s) is fetched and displayed.
- **Expected Output:**
  - The program correctly fetches and prints the records corresponding to the Zip Codes matching the results from 2.7.1.
- **Results**: **The test passed successfully.**

```
1. Print CSV Records
2. Print LRF Records
3. Convert CSV to LRF
4. Read Header Record
5. Write (Update) Header Record
6. Run State Extremes Report
7. Search for Zip Code Records by Index
Enter your choice: 7
Enter the LRF file name to search in: us_postal_codes_ROWS_RANDOMIZED
Enter Zip Code flags (e.g., -Z56301 -Z12345), separated by spaces: -Z56303
-Z56301 -Z12345
Record for Zip Code 56303:
Zip_Code: 56303 | Place_Name: Saint Cloud | State: MN | County: Stearns |
Lat: 45.5713 | Long: -94.2036 |
Record for Zip Code 56301:
Zip_Code: 56301 | Place_Name: Saint Cloud | State: MN | County: Stearns |
Lat: 45.541 | Long: -94.1819 |
Record for Zip Code 12345:
Zip_Code: 12345 | Place_Name: Schenectady | State: NY | County: Schenectady
| Lat: 42.8142 | Long: -73.9396 |
```

2.7.3. Test Case: Search us_postal_codes_COLUMNS_RANDOMIZED.lfr Zip Code Records by Index

- **Function:** `searchZipCodeRecords(const std::string& lrfFilename)`
- **Purpose:** Ensure that Zip Code records can be searched by using an index file.
- **Steps:**
  - Select “Search for Zip Code Records by Index” by inputting 7
  - Input a valid LRF file.
  - Provide one or more Zip Code flags as input.
  - Verify that the correct record(s) is fetched and displayed.
- **Expected Output:**
  - The program correctly fetches and prints the records corresponding to the Zip Codes matching the same data as 2.7.1 and 2.7.2 but different column order.
- **Results**: **The test passed successfully.**

```
1. Print CSV Records
2. Print LRF Records
3. Convert CSV to LRF
4. Read Header Record
5. Write (Update) Header Record
6. Run State Extremes Report
7. Search for Zip Code Records by Index
Enter your choice: 7
Enter the LRF file name to search in: us_postal_codes_COLUMNS_RANDOMIZED
Enter Zip Code flags (e.g., -Z56301 -Z12345), separated by spaces: -Z56303
-Z56301 -Z12345
Record for Zip Code 56303:
ZipCode: 56303 | Lat: 45.5713 | State: MN | County: Stearns | PlaceName:
Saint Cloud | Long: -94.2036 |
Record for Zip Code 56301:
ZipCode: 56301 | Lat: 45.541 | State: MN | County: Stearns | PlaceName:
Saint Cloud | Long: -94.1819 |
Record for Zip Code 12345:
ZipCode: 12345 | Lat: 42.8142 | State: NY | County: Schenectady |
PlaceName: Schenectady | Long: -73.9396 |
```

2.7.4. Test Case: Search us_postal_codes_ROWS_COLUMNS_RANDOMIZED.lfr Zip Code Records by Index

- **Function:** `searchZipCodeRecords(const std::string& lrfFilename)`
- **Purpose:** Ensure that Zip Code records can be searched by using an index file.
- **Steps:**
  - Select “Search for Zip Code Records by Index” by inputting 7
  - Input a valid LRF file.
  - Provide one or more Zip Code flags as input.
  - Verify that the correct record(s) is fetched and displayed.
- **Expected Output:**
  - The program correctly fetches and prints the records corresponding to the Zip Codes matching the same data as 2.7.3 but with the different randomized columns
- **Results**: **The test passed successfully.**

```
1. Print CSV Records
2. Print LRF Records
3. Convert CSV to LRF
4. Read Header Record
5. Write (Update) Header Record
6. Run State Extremes Report
7. Search for Zip Code Records by Index
Enter your choice: 7
Enter the LRF file name to search in:
us_postal_codes_ROWS_COLUMNS_RANDOMIZED
Enter Zip Code flags (e.g., -Z56301 -Z12345), separated by spaces: -Z56303
-Z56301 -Z12345
Record for Zip Code 56303:
ZipCode: 56303 | State: MN | Long: -94.2036 | County: Stearns | PlaceName:
Saint Cloud | Lat: 45.5713 |
Record for Zip Code 56301:
ZipCode: 56301 | State: MN | Long: -94.1819 | County: Stearns | PlaceName:
Saint Cloud | Lat: 45.541 |
Record for Zip Code 12345:
ZipCode: 12345 | State: NY | Long: -73.9396 | County: Schenectady |
PlaceName: Schenectady | Lat: 42.8142 |
```

…

3. Test Execution Procedure

1. **Prepare test environment:**
    - Prepare CSV and LRF files for testing, ensuring that they have different field types and record sizes.
    - Create corresponding `.idx` index files for Zip Code searches.
2. **Test Interaction:**
    - Manually interact with the program through the console to simulate user input for each of the test cases.
3. **Error Handling:**
    - Test invalid inputs, such as non-existent files or invalid file types, to ensure proper error messages are displayed.
    - Ensure that error handling is robust for situations like missing or corrupted records.

---

4. Expected Results

- For each test case, the program should:
    - Correctly process and display data from CSV, LRF, and header records.
    - Accurately write and update header records.

- Properly search for Zip Code records using index files.
- Handle errors gracefully, such as missing files or invalid inputs.

---

5. Conclusion

- **Summary:** This document outlines the tests for each function in the main program, focusing on the handling of CSV, LRF, and header records. Testing will ensure that the software works as intended and can handle real-world file operations.
- **Next Steps:** After running the tests, address any issues discovered and optimize the program for efficiency and accuracy.

…